

۱. Locust چیست و چرا به آن نیاز داریم؟

تصور کن یک رستوران کوچک داری که روزی ۱۰ مشتری دارد. آشپزخانه‌ات به راحتی پاسخگوست. حالا ناگهان ۱۰۰۰ مشتری همزمان وارد می‌شوند. آیا آشپزخانه‌ات می‌تواند بدون اینکه از هم بپاشد، غذای همه را به موقع تحویل دهد؟

در دنیای نرم‌افزار، تست بار (Load Testing) دقیقاً همین کار را می‌کند: به سیستم فشار می‌آورد تا ببینیم تحت بار سنگین چگونه رفتار می‌کند. Locust یک ابزار محبوب و رایگان پایتون برای انجام تست بار است.

ویژگی‌های Locust:

- اسکریپت‌ها را با پایتون خالص می‌نویسی (بدون زبان خاص)
- رابط کاربری تحت وب برای شروع تست و مشاهده نمودارها
- قابلیت شبیه‌سازی هزاران کاربر همزمان
- گزارش‌های زنده از RPS، زمان پاسخ، درصد خطا

۲. مفاهیم کلیدی Locust

User (کاربر مجازی)

هر کاربر مجازی (User) یک انسان را شبیه‌سازی می‌کند. این کاربر می‌تواند عملیات‌هایی مثل جستجوی کتاب را انجام دهد. Locust این کاربران را به صورت همزمان اجرا می‌کند.

Task (وظیفه)

هر کاری که کاربر انجام می‌دهد (مثلاً درخواست GET به `/search/`). می‌توان وزن (weight) داد تا بعضی کارها بیشتر از بقیه انجام شوند.

wait_time

مدت زمان انتظار بین دو کار متوالی یک کاربر. این انتظار رفتار انسان را شبیه‌سازی می‌کند (انسان‌ها پشت سر هم کلیک نمی‌کنند).

(RPS (Requests Per Second

تعداد درخواست‌هایی که سرور در هر ثانیه پاسخ می‌دهد. معیار اصلی توان عملیاتی سرور.

Response Time (زمان پاسخ)

مدت زمانی که طول می‌کشد تا سرور به یک درخواست پاسخ دهد. معمولاً با میلی‌ثانیه اندازه‌گیری می‌شود.

Failure Rate (نرخ خطا)

درصد درخواست‌هایی که با خطا مواجه می‌شوند (مثلاً 500 یا timeout).

۳. ساختار یک فایل locustfile.py

```
import random
```

```
from locust import HttpUser, task, between
```

HttpUser: کلاس پایه برای کاربرانی که درخواست HTTP می‌فرستند.

task: دکوراتوری که یک متد را به یک وظیفه تبدیل می‌کند.

between: تابعی که می‌گوید کاربر بین هر کار، چه مدت تصادفی صبر کند.

```
class BookUser(HttpUser:(
```

```
    wait_time = between(1, 3)  # هر کاربر بین ۱ تا ۳ ثانیه صبر می‌کند
```

```
search_terms = ["the", "and", "of", "war", "ring",  
"love", "great", "killl["
```

چرا `wait\_time` مهم است؟

اگر کاربران بدون وقفه درخواست بفرستند، رفتار ربات را خواهند داشت نه انسان واقعی. این بازه تصادفی، الگوی ترافیک واقعی را شبیه‌سازی می‌کند.

(وزن ۳: این وظیفه ۳ برابر بیشتر اجرا می‌شود 3 task @

```
def search_books(self:(  
    q = random.choice(self.search_terms(  
    page = random.randint(1, 3(  
    self.client.get)  
        f"/books/search?q={q}&page={page}&page_size=5,"  
        name="/books/search,"  
(
```

@task(3): یعنی این متد ۳ برابر سنگین‌تر از یک task معمولی وزن دارد. در سیستم واقعی، جستجو بیشتر از لیست کردن همه کتاب‌ها استفاده می‌شود.

self.client.get(...): یک درخواست GET می‌فرستد. این `client` مثل `requests` کتابخانه پایتون عمل می‌کند.

name="/books/search": اسم گروهی برای این درخواست. در نمودارها همه درخواست‌ها با `q`های مختلف زیر همین اسم گروه‌بندی می‌شوند.

python`

```
@ task(1(
```

```

def list_books(self:(
    self.client.get("/books", name="/books("

@ task(2(
def search_authors(self:(
    q = random.choice(["orwell", "tolkien", "lee", "austen("
    self.client.get(f"/authors/search?q={q}",
name="/authors/search("
'''

```

وظایف دیگر با وزن‌های مختلف. مجموع وزن‌ها: $3+1+2=6$. بنابراین احتمال اجرای `search_books` برابر $6/3 = 50\%$ است.

۴. اجرای تست بار

مرحله ۱: سرور را اجرا کن

```
make run
```

یا `uvicorn app.main:app --host 0.0.0.0 --port 8000`

مرحله ۲: Locust را اجرا کن

```
make locust
```

یا `locust -f tests/locustfile.py --host=http://localhost:8000`

مرحله ۳: باز کردن رابط کاربری

مرورگر را باز کن و به `http://localhost:8089` برو. صفحه‌ای می‌بینی که از تو می‌خواهد:

Number of users: تعداد کاربران همزمان (مثلاً ۱۰۰)

Spawn rate: نرخ افزایش کاربران در ثانیه (مثلاً ۱۰ کاربر در ثانیه)

Host: همان `http://localhost:8000` (پیش‌فرض از دستور)

دکمه Start Swarming را بزن.

مرحله ۴: مشاهده نمودارها

در تب Charts می‌توانی:

Total Requests per Second: تعداد کل درخواست‌ها در ثانیه (RPS)

Response Times: میانگین، صدک ۵۰ام (Median)، صدک ۹۵ام زمان پاسخ

Number of Users: رشد کاربران

در تب Failures خطاها را می‌بینی.

۵. تحلیل نتایج (مثال عملی)

فرض کن تست را با ۱۰۰ کاربر همزمان اجرا کردی و این نتایج را گرفتی:

معیار	بدون کش	با Redis Cache
RPS	۴۵	۳۲۰
میانگین زمان پاسخ	۲۱۰۰ ms	۳۱۰ ms
صدک ۹۵ام زمان پاسخ	۴۸۰۰ ms	۶۵۰ ms
نرخ خطا	۰٪	۰٪

تفسیر:

- RPS بالاتر با کش: یعنی سرور می‌تواند ۳۲۰ درخواست در ثانیه را پاسخ دهد، در حالی که قبلاً فقط ۴۵ درخواست را جواب می‌داد. این یعنی توان عملیاتی ۷ برابر شده است.
- زمان پاسخ کمتر: کاربر بدون کش باید ۲.۱ ثانیه صبر می‌کرد، اما حالا فقط ۰.۳۱ ثانیه صبر می‌کند. تجربه کاربری عالی می‌شود.
- صدک ۹۵٪: یعنی ۹۵٪ کاربران در کمتر از ۶۵۰ میلی‌ثانیه پاسخ می‌گیرند. این عدد برای سیستم‌های واقعی حیاتی است.

۶. افزایش حجم داده و تکرار تست

طبق دستور استاد، باید این مراحل را انجام دهی:

- حجم داده را زیاد کن (با اسکریپت، داده‌های Mock بیشتری به دیتابیس اضافه کن حداقل ۱۰,۰۰۰ کتاب). می‌تونی از AI بخواهی یک اسکریپت پایتون برای insert تصادفی بنویسد.
- بدون کش تست بار بگیر و نتایج را ذخیره کن (عکس از نمودار).
- کش را فعال کن (همین Redis که داری).
- دوباره تست بار بگیر و نتایج را مقایسه کن.
- تفسیر کن: ببین با افزایش حجم داده، RPS چطور افت می‌کند و کش چطور نجات‌بخش است.

۷. نکات پیشرفته و Best Practice

تست با کوئری‌های تصادفی و متفاوت

در کد Locust از `random.choice` استفاده کردیم. اگر همیشه یک کوئری ثابت تست شود، کش همه را پاسخ می‌دهد و نتایج گول‌زننده است. تنوع کوئری، سناریوی واقعی را شبیه‌سازی می‌کند.

تست افزایش تدریجی کاربران

می‌توانی در Locust نرخ افزایش کاربران را کم کنی (مثلاً ۵ کاربر در ثانیه) تا ببینی سرور در چه نقطه‌ای شروع به خطا دادن می‌کند. این نقطه «نقطه شکست» (Breaking Point) نام دارد.

اندازه‌گیری تأثیر Connection Pool

با تغییر `DB_MIN_CONN` و `DB_MAX_CONN` تست را تکرار کن. بین آیا RPS تغییر می‌کند یا نه. این بهینه‌سازی را می‌توانی در گزارش ارائه دهی.

۸. خلاصه

Locust ابزاری ساده اما قدرتمند برای سنجش توان سرور است. با نوشتن چند خط پایتون می‌توانی هزاران کاربر را شبیه‌سازی کنی و بفهمی سیستمات زیر بار چقدر دوام می‌آورد. در پروژه هفته دوم، از Locust استفاده می‌کنی تا تأثیر کش Redis را به صورت عددی نشان دهی و این یک مهارت بسیار ارزشمند برای رزومه و پروژه‌های واقعی است.